

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Naveen Kumar S	Reg.No.:	192421030
Department:	B Tech IT	Date:	25.08.2026

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Real-Time Embedded I/O Communication Mechanism

A real-time embedded I/O communication mechanism can be designed using **memory-mapped I/O, vectored interrupt handling, and DMA** to satisfy the requirement that CPU utilization remains below 15%. Since multiple sensors generate interrupts every 1 ms, continuously processing every sensor operation directly by the CPU would create a high interrupt load. Therefore, each sensor can be assigned a separate interrupt vector so that the interrupt controller can immediately identify the source and transfer control to the corresponding interrupt service routine (ISR). The ISR should perform only essential operations, such as reading the sensor status and initiating a DMA transfer, and then return quickly. **DMA (Direct Memory Access)** can transfer sensor data directly to main memory without continuous CPU involvement. Memory-mapped I/O is preferred because device registers can be accessed using normal memory instructions, providing fast and simple communication with the sensors. The combination of vectored interrupts, short ISRs, and DMA reduces CPU overhead and helps maintain CPU utilization below 15% while providing low interrupt latency and reliable real-time performance.

2. High-Speed I/O Subsystem

A high-speed I/O subsystem can be designed using an **NVMe storage device connected through PCIe**, along with DMA-based data transfers, to achieve a throughput greater than 3 GB/s. NVMe is specifically designed for high-speed solid-state storage and uses the PCIe interface to provide high bandwidth and low latency. The NVMe controller communicates with the CPU through submission and completion queues, allowing multiple I/O requests to be processed efficiently. Instead of requiring the CPU to copy every block of storage data, a **DMA mechanism** transfers data directly between the NVMe device and main memory. The CPU is responsible mainly for creating I/O commands, managing queues, and handling completion notifications rather than performing bulk data movement. Interrupts can notify the CPU when a

DMA operation has completed or when an error occurs. This architecture significantly reduces CPU overhead and allows the storage subsystem to achieve high throughput while maintaining efficient processor utilization.

3. Multi-Device Communication Architecture

A multi-device communication architecture can be constructed to support **six USB peripherals and two high-speed Thunderbolt devices** while preventing interrupt starvation and ensuring fair bus access. The USB devices can be connected through a USB host controller or suitable USB hubs, while the two high-speed devices can communicate through Thunderbolt controllers. Hardware interrupts should be used for time-critical events that require immediate CPU attention, such as device errors or completion of important transfers. **Software interrupts** can be used for deferred processing and background operations so that lengthy processing does not occur inside hardware interrupt service routines. To avoid interrupt starvation, devices should be assigned appropriate priority levels and interrupt processing should be carefully scheduled. For bus access, a **fair arbitration technique such as round-robin arbitration** can be used so that each device receives an opportunity to use the shared communication resources. High-priority devices can still receive faster service when necessary, while lower-priority devices are prevented from waiting indefinitely. DMA can also be used for large data transfers to reduce CPU involvement and improve overall system throughput.

4. Data Acquisition System

A data acquisition system receiving a continuous data stream of **500 MB/s** should use **memory-mapped I/O and DMA** to minimize CPU intervention. The acquisition device can expose its control and status registers through memory-mapped I/O, allowing the CPU to access them using normal memory instructions. Since the incoming data rate is very high, using interrupt-driven I/O for every individual data item would generate a large number of interrupts and consume significant CPU time. Instead, DMA can transfer large blocks of data directly from the acquisition device into preallocated memory buffers. The CPU can configure the DMA controller initially and then perform other tasks while the data transfer takes place. An interrupt can be generated only after a complete buffer has been filled, rather than for every data item. **DMA therefore provides much better throughput and lower CPU utilization** than interrupt-driven I/O for a continuous 500 MB/s stream. Double buffering or circular buffering can also be used so that one buffer is being filled while the CPU processes another buffer, preventing data loss and maintaining continuous acquisition.

5. Interrupt Handling Mechanism

An interrupt handling mechanism supporting **nested interrupts, vectored interrupts, and software interrupts** can be designed to achieve a maximum interrupt latency of less than **10 μ s**. High-priority devices should use vectored hardware interrupts, allowing the processor to directly locate the appropriate interrupt service routine without searching through all possible interrupt sources. Interrupts can be assigned different priority levels, with critical devices receiving the highest priority. **Nested interrupt handling** allows a high-priority interrupt to interrupt a lower-priority ISR, ensuring that urgent events are serviced immediately. To satisfy the 10 μ s latency requirement, high-priority ISRs should be extremely short and should perform only critical operations, while lengthy processing should be postponed. Software interrupts can then be used for deferred or background tasks after the urgent hardware event has been handled. Proper priority management, fast interrupt-vector lookup, short ISRs, and nested interrupt support together provide quick response to critical devices while allowing background tasks to execute efficiently.